

关于第一次大作业的若干提示

赵银亮

2026.4.3

示例针对 fact 程序的基于 σ -DFA(<Q2ARM 左列>)的指令选择实现及测试过程，分为四步：构建 σ -DFA 并程序实现，能以任务程序作为输入，输出 ARM 程序；手工补全为完整汇编程序；汇编、链接、运行；用 GDB 查看运行过程。

已知任务程序：

```
fact@code=[LABEL l0; t1=2; IF n<t1 THEN l1 ELSE l2; LABEL l1; RETURN a;  
GOTO l3; LABEL l2; t2=n-1; t3=n*a; n=t2; a=t3; GOTO l0; LABEL l3;]
```

新版 QTAC 及 Q2ARM 见文档末尾。

第一步为构建 σ -DFA 并程序实现。

如以下提示，分为 4 遍完成。

第 1 遍输出：

```
(KEY,LABEL) (1,l0) (;,_) (temp,t1) (=,_) (k,2) (;,_) (KEY,IF) (d,n) (rop,  
<) (temp,t1) (KEY,THEN) (1,l1) (KEY,ELSE) (1,l2) (;,_) (KEY,LABEL) (1,l  
1) (;,_) (KEY,RETURN) (d,a) (;,_) (KEY,GOTO) (1,l3) (;,_) (KEY,LABEL)  
(1,l2) (;,_) (temp,t2) (=,_) (d,n)(aop,-) (k,1) (;,_) (temp,t3)(=,_) (d,  
n) (aop,*) (d,a) (;,_) (d,n)(=,_) (temp,t2) (;,_) (d,a)(=,_) (temp,t3) (;,  
_) (KEY,GOTO) (1,l0) (;,_) (KEY,LABEL) (1,l3) (;,_)
```

第 2 遍输出：

```
(T_LABEL,(1,l0))  
(T_MOVE,a is k,(temp,t1),(k,2))  
(T_IF_LABEL,l1=l3,(d,n),(rop,<),(temp,t1),(1,l1),(1,l2))  
(T_RETURN,a is q,(d,a))  
(T_GOTO,(1,l3))  
(T_LABEL,(1,l2))  
(T_AOP,a is k,(temp,t2),(d,n),(aop,-),(k,1))  
(T_AOP,a is q,(temp,t3),(d,n),(aop,*),(d,a))  
(T_MOVE,a is q,(d,n),(temp,t2))  
(T_MOVE,a is q,(d,a),(temp,t3))  
(T_GOTO,(1,l0))  
(T_LABEL,(1,l3))
```

第 3 遍输出：

l0:

```

MOV Xt1, #2;
CMP Xn, Xt1; B.GE l2; l1:
MOV X0,Xa; RET;
B l3;
l2:
SUB Xt2,Xn,#1;
MUL Xt3,Xn,Xa;
MOV Xn, Xt2;
MOV Xa, Xt3;
B l0;
l3:

```

第 4 遍输出：其中 $X_{t1}=X_9$, $X_n=X_{10}$, $X_a=X_{11}$, $X_{t2}=X_{12}$, $X_{t3}=X_{13}$, 其它不变。

```

l0:
MOV X9, #2;
CMP X10, X9; B.GE l2; l1:
MOV X0, X11; RET;
B l3;
l2:
SUB X12,X10,#1;
MUL X13,X10,X11;
MOV X10,X12;
MOV X11,X13;
B l0;
l3:

```

第二步手工添加代码形成完整的 ARM 程序。

```

.global _start

.section .text

/* =====
   主程序入口
   ===== */
_start:
    /* 1. 准备参数 */
    MOV X10, #5        // 对应代码中的 X10 (n)
    MOV X11, #1        // 对应代码中的 X11 (a)

    /* 2. 调用函数 */
    BL my_func

```

```

/* 3. 退出程序 (Linux 系统调用) */
// 如果程序正常返回, X0 中会有结果
MOV X8, #93          // sys_exit
SVC #0

/* =====
   第一步生成的 ARM 代码 (原封不动)
   ===== */
my_func:
I0:
    MOV X9, #2;
    CMP X10, X9;
    B.GE I2;

I1:
    MOV X0, X11;
    RET;
    B I3;          // <--- 逻辑残留: 死代码, RET 后无法执行

I2:
    SUB X12,X10,#1;
    MUL X13,X10,X11;
    MOV X10, X12;
    MOV X11, X13;
    B I0;

I3:
    B I3;          // <--- 逻辑残留: 作为程序的“陷阱”, 以后有用处 (老师注)

```

第三步：运行并查看结果

1. 汇编

```
aarch64-linux-gnu-as -o program.o program.s
```

2. 链接

```
aarch64-linux-gnu-ld -o program program.o
```

3. 运行并查看返回码 (应该是 120)

```
./program
```

```
echo $?
```

第四步：采用 GDB 了解执行过程

要使用 GDB 查看这段 ARM64 汇编程序的运行过程，我们需要通过设置断点、单步执行和查看寄存器来观察 X10（计数器）和 X11（累积结果）的变化。

由于你的程序逻辑中包含了一个“死循环” (I3)，调试时我们会特别关注程序是如何跳转到那里并卡住的。

以下是完整的操作步骤指南：

1. 重新编译（带调试信息）

首先，必须加上 `-g` 参数重新编译，这样 GDB 才能识别代码行号。

```
aarch64-linux-gnu-as -g -o program.o program.s
```

```
aarch64-linux-gnu-ld -g -o program program.o
```

2. 启动 GDB

在终端中输入以下命令启动调试：

```
gdb program
```

3. 调试核心流程（GDB 命令速查）

进入 GDB 界面后（提示符为 `(gdb)`），你可以按照以下步骤操作：

第 A 步：找到入口并设置断点

我们首先在 `_start`（主程序入口）和 `my_func`（你的函数入口）处设置断点，防止程序一运行就结束。

． 调试核心流程（GDB 命令速查）

进入 GDB 界面后（提示符为 `(gdb)`），你可以按照以下步骤操作：

第一步：找到入口并设置断点

我们首先在 `_start`（主程序入口）和 `my_func`（你的函数入口）处设置断点，防止程序一运行就结束。

第 B 步：运行程序

输入 `run` (或简写 `r`) 启动程序。程序会在遇到第一个断点时暂停。

```
(gdb) run
```

第 C 步：单步跟踪（观察寄存器变化）

这是最关键的一步。当程序停在 `my_func` 时，使用 `nexti` (next instruction) 来一行一行地执行汇编指令，并使用 `info registers` 查看寄存器状态。

操作演示：

查看初始状态：

当程序停在 `my_func` 时，输入：

```
(gdb) info registers X10 X11
```

你应该看到 X10 是 5，X11 是 1。

单步执行循环：

不断输入 `nexti` (或 `ni`) 并观察寄存器。

当执行到 `l2` 标号附近的代码时 (`SUB, MUL`)，再次查看寄存器：

```
(gdb) info registers X10 X11 X12 X13
```

观察点：你会发现 `X10` 每次减 1，`X11` 变成了累乘的结果。

观察跳转逻辑：

当 `X10` 变为 1 时，执行 `CMP X10, X9` 和 `B.GE l2`。

此时条件不满足 (1 不大于等于 2)，程序会跳转到 `l1`。

观察返回与“残留逻辑”：

程序执行到 `l1` 的 `RET` 指令。

执行 `RET` 后，程序会返回到主程序的 `_start` 处继续执行退出系统调用。

注意：你代码中的 `l3` 标号 (死循环) 实际上不会被执行到，因为 `RET` 指令已经让函数返回了。`B l3` 是死代码。

常用 GDB 命令速查表

命令 (及简写)	功能说明	针对本程序的用途
<code>break (b)</code>	设置断点	<code>b my_func</code> ：在函数开始处停下，准备观察。
<code>run (r)</code>	运行程序	开始执行程序，直到遇到断点。
<code>nexti (ni)</code>	单步执行 (汇编级)	最常用。一步一步看 <code>SUB</code> 和 <code>MUL</code> 怎么改变数值。
<code>info reg</code>	查看寄存器	<code>info reg X10 X11</code> ：专门看你的变量变化。
<code>display</code>	自动显示	<code>display/10i \$pc</code> ：每次暂停时，自动显示接下来的 10 行汇编代码。
<code>continue (c)</code>	继续运行	直接跳到下一个断点 (如果你想跳过繁琐的循环)。
<code>quit (q)</code>	退出 GDB	调试结束。

实战示例 (模拟对话)

```
(gdb) b my_func           # 在函数入口打断点
```

```
Breakpoint 1 at 0x400078
```

```
(gdb) r                   # 运行
```

```
Starting program: ./program
```

```
Breakpoint 1, my_func () at program.s:15
```

```
15          l0:
```

```
(gdb) info reg X10 X11     # 查看初始值
```

```
X10          0x5          # n = 5
```

```

X11          0x1          # a = 1
(gdb) ni      # 执行 MOV X9, #2
(gdb) ni      # 执行 CMP X10, X9
(gdb) ni      # 执行 B.GE l2 (跳转发生)
(gdb) ni      # 现在在 l2 标号处
(gdb) ni      # 执行 SUB X12, X10, #1
(gdb) info reg X12      # 查看 X12
X12          0x4          # 5 - 1 = 4
(gdb) ni      # 执行 MUL X13, X10, X11
(gdb) info reg X13      # 查看 X13
X13          0x5          # 5 * 1 = 5
...           # 继续 ni 直到循环结束

```

通过这种方式，你可以亲眼看到数据是如何在寄存器之间流动和计算的。

指令模板 Q2ARM

赵银亮
西安交通大学
2026.4.1

$\mathcal{L}$ (QTAC 指令模板)	ARM64 指令生成模式
$\langle T_RETURN \rangle \rightarrow \text{RETURN } a;$	$(a \text{ is } q) \text{ MOV } X_0, X_a; \text{ RET};$ $(a \text{ is } k) \text{ MOV } X_0, \#k; \text{ RET};$
$\langle T_PAR_CALL \rangle \rightarrow (\text{PAR } a;)\{0,7\} q = \text{CALL } d, k;$	$\text{MOV } X_0, X_{a1}; \dots; \text{MOV } X_k, X_{ak};$ $\text{BL } d; \text{MOV } X_q, X_0;$
$\langle T_GOTO \rangle \rightarrow \text{GOTO } b;$	$(b \text{ is } l) \text{ B } l;$ $(b \text{ is } q) \text{ BR } X_q;$
$\langle T_LABEL \rangle \rightarrow \text{LABEL } l;$	$l:$
$\langle T_IF_LABEL \rangle \rightarrow \text{IF } q \langle \text{rop} \rangle a \text{ THEN } l_1 \text{ ELSE } l_2;$ $\text{LABEL } l_3;$	$(l_3 = l_2) \text{ CMP } X_q, X_a; \langle \text{Jrop} \rangle l_1; l_2;$ $(l_3 = l_1) \text{ CMP } X_q, X_q; \langle \text{Jrop}' \rangle l_2; l_1;$
$\langle T_IF \rangle \rightarrow \text{IF } q \langle \text{rop} \rangle a \text{ THEN } l_1 \text{ ELSE } l_2;$	$\text{CMP } X_q, X_a; \langle \text{Jrop} \rangle l_1; \text{B } l_2;$
$\langle T_LDR0 \rangle \rightarrow q = M[b];$	$(b \text{ is } q) \text{ LDR } X_q, [X_b];$ $(b \text{ is } l) \text{ LDR } X_q, b;$
$\langle T_LDR2 \rangle \rightarrow q_1 = q_2 + a; q_3 = M[q_1];$	$(q_1 = q_2 \wedge a \text{ is } k) \text{ LDR } X_{q3}, [X_{q1}, \#k]!;$ $(q_1 \neq q_2 \wedge a \text{ is } q \wedge q^{\text{last}}) \text{ LDR } X_{q3}, [X_{q1}, X_a];$ $(q_1 \neq q_2 \wedge a \text{ is } k \wedge q^{\text{last}}) \text{ LDR } X_{q3}, [X_{q1}, \#a];$
$\langle T_STR0 \rangle \rightarrow M[b] = q;$	$(b \text{ is } q) \text{ STR } X_q, [X_b];$ $(b \text{ is } l) \text{ ADR } \langle X_{\text{crs}} \rangle, b; \text{STR } X_q, [X_{\text{crs}}];$
$\langle T_STR2 \rangle \rightarrow q_1 = q_2 + a; M[q_1] = q_3;$	$(q_1 = q_2 \wedge a \text{ is } k) \text{ STR } X_{q3}, [X_{q1}, \#k]!;$

	$(q_1 \neq q_2 \wedge a \text{ is } q \wedge q^{\text{last}}) \text{ STR } X_{q_3}, [X_{q_1}, X_a];$ $(q_1 \neq q_2 \wedge a \text{ is } k \wedge q^{\text{last}}) \text{ STR } X_{q_3}, [X_{q_1}, \#a];$
$\langle \text{T_AOP} \rangle \rightarrow q_1 = q_2 \langle \text{aop} \rangle a;$	$(a \text{ is } q) \langle \text{Xaop} \rangle X_{q_1}, X_{q_2}, X_a;$ $(a \text{ is } k) \langle \text{Kaop} \rangle X_{q_1}, X_{q_2}, \#a;$
$\langle \text{T_MOVE} \rangle \rightarrow q = a;$	$(a \text{ is } q) \text{ MOV } X_q, X_a;$ $(a \text{ is } k) \text{ MOV } X_q, \#a;$
$\langle \text{Jrop} \rangle$: B.EQ、B.NE、B.GE、B.GT、B.LE、B.LT $\langle \text{Jrop}' \rangle$: B.NE、B.EQ、B.LT、B.LE、B.GT、B.GE $\langle \text{Xaop} \rangle$: ADD、SUB、MUL、SDIV $\langle \text{Kaop} \rangle$: ADD、SUB $[X_{\text{crs}}]$ 虚拟寄存器, 一次赋值仅一次使用 t 是临时变量, 即 $\langle \text{temp} \rangle$; t^{last} 与 t 没有区别; 如果 x 是虚拟寄存器, 那么 X_x 就是 x, 否则 X_x 是虚拟寄存器。正体 X 开头的指物理寄存器。	

中间语言 QTAC

赵银亮
西安交通大学
2026.4.1

$d@code=[\langle QTAC-ins-list \rangle]$
 $\langle QTAC-ins-list \rangle \rightarrow \langle QTAC-ins \rangle ; | \langle QTAC-ins-list \rangle \langle QTAC-ins \rangle ;$
 $\langle QTAC-ins \rangle \rightarrow LABEL\ l | GOTO\ l | IF\ q\ \langle rop \rangle\ a\ THEN\ l\ ELSE\ l$
 $q \rightarrow d | \langle temp \rangle$
 $a \rightarrow q | k$
 $\langle QTAC-ins \rangle \rightarrow PAR\ a\ | \ q = CALL\ d, k | RETURN\ a$
 $\langle QTAC-ins \rangle \rightarrow q = q\ \langle aop \rangle\ a\ | \ q = a$
 $\langle QTAC-ins \rangle \rightarrow q = M[b] | M[b] = q$
 $b \rightarrow q | l$
 $\langle TAC-ins \rangle \rightarrow q = \&b | NOP | q = CONVERT\ q, t$

k 是立即数, $k \in [0, 4095]$; t 为 INT 或 FLOAT; l 为中间语言标号名, 形式为 li ; $\langle temp \rangle$ 为临时变量名, 形式为 ti 。 $\langle aop \rangle$ 与 $\langle rop \rangle$ 分别是双目算术和关系运算符, 优先级前者高于后者。

$d \rightarrow [a-z]^+$

$\langle temp \rangle \rightarrow T_ [0-9]^+$

$l \rightarrow L_ [0-9]^+$

$k \rightarrow [0-9]^+$

$\langle aop \rangle \rightarrow + | - | * | /$

$\langle rop \rangle \rightarrow < | <= | > | >= | == | !=$

KEY $\rightarrow LABEL | GOTO | IF | THEN | ELSE | PAR | CALL | RETURN | RETURN | ASG |$

AOP $| LOAD | STORE | ADDR | NOP | CONVERT$

$\langle QTAC\ 程序 \rangle \rightarrow (\langle QTAC-ins \rangle ;)^+$